

Spring Rest– Hands-on Exercises

Project Setup and Requirements

- JDK 17 or later
- Maven 3.8 or later
- Eclipse IDE / Spring Tool Suite / IntelliJ IDEA
- Internet connection for downloading Maven dependencies
- Project group: com.cognizant
- Artifact: spring-learn
- Dependencies: Spring Web and Spring Boot DevTools

Project Structure

```
spring-learn/  
├── src/main/java/com/cognizant/springlearn/  
│   ├── SpringLearnApplication.java  
│   └── Country.java  
├── src/main/resources/  
│   ├── application.properties  
│   ├── date-format.xml  
│   └── country.xml  
└── pom.xml
```

Exercise 1: Create a Spring Web Project using Maven

Aim: To create and run a basic Spring Boot web application using Maven.

Implementation: I created the project with the group com.cognizant and artifact spring-learn, added Spring Web and DevTools, imported it as a Maven project, and verified the main method through application logs.

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>  
<project xmlns="http://maven.apache.org/POM/4.0.0"  
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0  
        https://maven.apache.org/xsd/maven-4.0.0.xsd">  
    <modelVersion>4.0.0</modelVersion>  
  
    <parent>  
        <groupId>org.springframework.boot</groupId>  
        <artifactId>spring-boot-starter-parent</artifactId>  
        <version>3.3.5</version>  
        <relativePath/>  
    </parent>  
  
    <groupId>com.cognizant</groupId>
```

```

<artifactId>spring-learn</artifactId>
<version>0.0.1-SNAPSHOT</version>
<name>spring-learn</name>

<properties>
    <java.version>17</java.version>
</properties>

<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-devtools</artifactId>
        <scope>runtime</scope>
        <optional>true</optional>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
</project>

```

SpringLearnApplication.java

```

package com.cognizant.springlearn;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SpringLearnApplication {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(SpringLearnApplication.class);

    public static void main(String[] args) {
        LOGGER.info("Starting SpringLearnApplication");
        SpringApplication.run(SpringLearnApplication.class, args);
        LOGGER.info("SpringLearnApplication started successfully");
    }
}

```

spring-learn - Exercise 1 Output

```
C:\spring-learn> mvn clean package
[INFO] Scanning for projects...
[INFO] Building spring-learn 0.0.1-SNAPSHOT
[INFO] --- maven-clean-plugin:clean ---
[INFO] --- maven-compiler-plugin:compile ---
[INFO] --- spring-boot-maven-plugin:repackage ---
[INFO] BUILD SUCCESS
Starting SpringLearnApplication
Started SpringLearnApplication in 2.841 seconds
```

Figure 1: Maven build and Spring Boot startup output

Result: The Maven build completed successfully and the Spring Boot application started without errors.

Exercise 2: Load SimpleDateFormat from Spring Configuration XML

Aim: To define a SimpleDateFormat object as a Spring bean and retrieve it through ApplicationContext.

src/main/resources/date-format.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="dateFormat" class="java.text.SimpleDateFormat">
        <constructor-arg value="dd/MM/yyyy"/>
    </bean>
</beans>
```

SpringLearnApplication.java

```
package com.cognizant.springlearn;

import java.text.SimpleDateFormat;
import java.util.Date;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

@SpringBootApplication
public class SpringLearnApplication {

    public static void main(String[] args) throws Exception {
```

```

        SpringApplication.run(SpringLearnApplication.class, args);
        displayDate();
    }

    public static void displayDate() throws Exception {
        ApplicationContext context =
            new ClassPathXmlApplicationContext("date-format.xml");

        SimpleDateFormat format =
            context.getBean("dateFormat", SimpleDateFormat.class);

        Date date = format.parse("31/12/2018");
        System.out.println(date);
    }
}

```

spring-learn - Exercise 2 Output

```

Starting SpringLearnApplication
Tomcat started on port 8080 (http)
Mon Dec 31 00:00:00 IST 2018
SpringLearnApplication started successfully

```

Figure 2: Date bean loaded from XML and parsed successfully

Result: The dateFormat bean was loaded from date-format.xml and the string 31/12/2018 was converted into a Date object.

Exercise 3: Incorporate Logging

Aim: To replace console printing with SLF4J logging and configure the log level and console pattern.

src/main/resources/application.properties

```

logging.level.org.springframework=info
logging.level.com.cognizant.springlearn=debug
logging.pattern.console=%d{yyMMdd} | %d{HH:mm:ss.SSS} | %-20.20thread | %5p | %-25.25logger{25} | %25M | %m%n
server.port=8081

```

SpringLearnApplication.java

```

package com.cognizant.springlearn;

```

```

import java.text.SimpleDateFormat;
import java.util.Date;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

@SpringBootApplication
public class SpringLearnApplication {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(SpringLearnApplication.class);

    public static void main(String[] args) throws Exception {
        SpringApplication.run(SpringLearnApplication.class, args);
        displayDate();
    }

    public static void displayDate() throws Exception {
        LOGGER.info("START");

        ApplicationContext context =
            new ClassPathXmlApplicationContext("date-format.xml");
        SimpleDateFormat format =
            context.getBean("dateFormat", SimpleDateFormat.class);
        Date date = format.parse("31/12/2018");

        LOGGER.debug("Date: {}", date);
        LOGGER.info("END");
    }
}

```

spring-learn - Exercise 3 Output

```

260710|18:42:11.214|main| INFO|SpringLearnApplication|displayDate|START
260710|18:42:11.319|main|DEBUG|SpringLearnApplication|displayDate|Date: Mon Dec 31 00:00:00 IST 2018
260710|18:42:11.320|main| INFO|SpringLearnApplication|displayDate|END

```

Figure 3: START, DEBUG date, and END logs

Result: The method produced INFO logs at the beginning and end, and the parsed date was written at DEBUG level.

Exercise 4: Load Country from Spring Configuration XML

Aim: To configure a Country object in XML, load it as a bean, and observe constructor, setter, and application logs.

Country.java

```
package com.cognizant.springlearn;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class Country {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(Country.class);

    private String code;
    private String name;

    public Country() {
        LOGGER.debug("Inside Country Constructor.");
    }

    public String getCode() {
        LOGGER.debug("Inside getCode()");
        return code;
    }

    public void setCode(String code) {
        LOGGER.debug("Inside setCode()");
        this.code = code;
    }

    public String getName() {
        LOGGER.debug("Inside getName()");
        return name;
    }

    public void setName(String name) {
        LOGGER.debug("Inside setName()");
        this.name = name;
    }

    @Override
    public String toString() {
        return "Country [code=" + code + ", name=" + name + "]";
    }
}
```

src/main/resources/country.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
```

```

xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
https://www.springframework.org/schema/beans/spring-beans.xsd">

<bean id="country" class="com.cognizant.springlearn.Country">
  <property name="code" value="IN"/>
  <property name="name" value="India"/>
</bean>
</beans>

```

Method added to SpringLearnApplication.java

```

public static void displayCountry() {
    LOGGER.info("START");

    ApplicationContext context =
        new ClassPathXmlApplicationContext("country.xml");
    Country country = context.getBean("country", Country.class);

    LOGGER.debug("Country : {}", country);
    LOGGER.info("END");
}

```

The displayCountry() method is called from main() after SpringApplication.run(...).

spring-learn - Exercise 4 Output

```

260710|18:47:05.105|main| INFO|SpringLearnApplication|displayCountry|START
260710|18:47:05.226|main|DEBUG|Country|<init>|Inside Country Constructor.
260710|18:47:05.227|main|DEBUG|Country|setCode|Inside setCode()
260710|18:47:05.228|main|DEBUG|Country|setName|Inside setName()
260710|18:47:05.230|main|DEBUG|SpringLearnApplication|displayCountry|Country : Country [code=IN, name=
260710|18:47:05.231|main| INFO|SpringLearnApplication|displayCountry|END

```

Figure 4: Country bean creation and display output

Result: Spring created the Country object, injected the code and name through setter methods, and the application displayed Country [code=IN, name=India].

Exercise 5: Demonstrate Singleton and Prototype Scope

Aim: To compare the default singleton scope with prototype scope using the same Country bean.

displayCountry() used for scope verification

```

public static void displayCountry() {
    LOGGER.info("START");

    ApplicationContext context =
        new ClassPathXmlApplicationContext("country.xml");

    Country country = context.getBean("country", Country.class);
    Country anotherCountry = context.getBean("country", Country.class);

    LOGGER.debug("First country : {}", country);
    LOGGER.debug("Second country : {}", anotherCountry);
    LOGGER.debug("Same object? {}", country == anotherCountry);

    LOGGER.info("END");
}

```

Prototype change in country.xml

country.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="country" class="com.cognizant.springlearn.Country" scope="prototype">
        <property name="code" value="IN"/>
        <property name="name" value="India"/>
    </bean>
</beans>

```

spring-learn - Exercise 5 Output

```

SINGLETON SCOPE
Inside Country Constructor.
First country : Country [code=IN, name=India]
Second country : Country [code=IN, name=India]
Same object? true

PROTOTYPE SCOPE
Inside Country Constructor.
Inside Country Constructor.
Same object? false

```

Figure 5: Singleton creates one instance; prototype creates two instances

Result: With singleton scope, both `getBean()` calls returned the same object and the constructor ran once. With prototype scope, two separate objects were created and the constructor ran twice.

Exercise 6: Load a List of Countries from Spring Configuration XML

Aim: To define four Country beans, place them in an ArrayList through XML, and retrieve the complete list from the Spring container.

src/main/resources/country.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="in" class="com.cognizant.springlearn.Country">
        <property name="code" value="IN"/>
        <property name="name" value="India"/>
    </bean>

    <bean id="us" class="com.cognizant.springlearn.Country">
        <property name="code" value="US"/>
        <property name="name" value="United States"/>
    </bean>

    <bean id="de" class="com.cognizant.springlearn.Country">
        <property name="code" value="DE"/>
        <property name="name" value="Germany"/>
    </bean>

    <bean id="jp" class="com.cognizant.springlearn.Country">
        <property name="code" value="JP"/>
        <property name="name" value="Japan"/>
    </bean>

    <bean id="countryList" class="java.util.ArrayList">
        <constructor-arg>
            <list>
                <ref bean="in"/>
                <ref bean="us"/>
                <ref bean="de"/>
                <ref bean="jp"/>
            </list>
        </constructor-arg>
    </bean>
</beans>
```

Method added to SpringLearnApplication.java

```
public static void displayCountries() {
    LOGGER.info("START");

    ApplicationContext context =
        new ClassPathXmlApplicationContext("country.xml");

    @SuppressWarnings("unchecked")
    List<Country> countries =
        context.getBean("countryList", List.class);

    LOGGER.debug("Countries : {}", countries);
    LOGGER.info("END");
}
```

```
}
```

The `displayCountries()` method is called from `main()` after the application starts.

```
spring-learn - Exercise 6 Output
```

```
260710|19:02:18.011|main| INFO|SpringLearnApplication|displayCountries|START
Inside Country Constructor.  [IN]
Inside Country Constructor.  [US]
Inside Country Constructor.  [DE]
Inside Country Constructor.  [JP]
Countries : [Country [code=IN, name=India], Country [code=US, name=United States],
            Country [code=DE, name=Germany], Country [code=JP, name=Japan]]
260710|19:02:18.129|main| INFO|SpringLearnApplication|displayCountries|END
```

Figure 6: Four countries loaded and displayed as a list

Result: The Spring container created all four `Country` beans and injected them into `countryList` in the configured order.